

UNIVERSIDAD DE LA SABANA

Maestría en Arquitectura de Software
Diseño y Optimización de Bases de Datos — MAS 2026-3 G1G2

DOCUMENTO TÉCNICO DE IMPLEMENTACIÓN

Análisis arquitectónico y evaluación crítica

PostgreSQL (Supabase) + MongoDB (Atlas)

Plataforma: Ecommify — Grupo E25

Integrantes del equipo

Andres Camilo Meneses Ortega
David Hernando Monsalve Delima
Eduardo Trujillo Santos
Nicolás Cifuentes Barriga

Docente: Miguel Alfonso Varela — Bogotá D.C., 2026

1. Executive Summary

Ecommify implementa una **arquitectura híbrida de persistencia polígloa**: **MongoDB Atlas** gestiona el catálogo de productos (**1,000 documentos**) y las reseñas (**3,118 documentos**) con esquema flexible; **PostgreSQL (Supabase)** gestiona el módulo transaccional (órdenes, clientes, pagos) con garantías ACID. Este informe documenta la evaluación crítica de dicha arquitectura bajo criterios de rendimiento, escalabilidad y decisiones basadas en evidencia empírica, tomando como base las optimizaciones implementadas en la Unidad 5.

Métrica Clave	MongoDB Atlas	PostgreSQL Supabase	Arquitectura Híbrida
Colecciones / Tablas	products (1k) · reviews (3,118) · reviews_buckets (954)	orders · customers · order_items	Polígloa — cada BD en su rol
Reducción docsExamined (ESR)	1,000 → 328 (-67.2%) Q1 catálogo	N/A (índices B-tree, GIN, BRIN)	Índices correctos = factor dominante
Bucket Pattern (reviews)	3,118 docs → 954 buckets (-69.4%)	—	Analítica temporal 10x más rápida
Clasificación CAP	AP (catálogo + reviews)	CP (órdenes + pagos)	Híbrido intencional por módulo

Conclusión principal: La elección híbrida no es una concesión arquitectónica sino una decisión deliberada. MongoDB es superior para catálogo y analítica de reviews; PostgreSQL es indispensable para la lógica transaccional. Ninguna tecnología aislada cumpliría ambos requisitos con la misma eficiencia.

2. Introducción y Contexto

2.1 Problema de Negocio

Ecommify enfrenta el reto clásico de plataformas de e-commerce de mediana escala: un catálogo con atributos altamente variables por categoría (relojes vs calzado tienen especificaciones completamente distintas), combinado con un núcleo transaccional que requiere ACID estricto para garantizar que ningún pago quede en estado indeterminado. Una solución 100% relacional sufre de esquemas rígidos para el catálogo y ausencia de sharding nativo. Una solución 100% documental sacrifica las garantías transaccionales del flujo de pago (transacciones multi colección en MongoDB tienen overhead de 3-5x).

2.2 Objetivos del Proyecto

(1) Diseñar e implementar una arquitectura híbrida de persistencia poliglota asignando cada módulo a la BD que mejor satisface sus requisitos de consistencia, escalabilidad y flexibilidad de esquema. (2) Optimizar el rendimiento con indexación avanzada (ESR, Attribute, Bucket, BRIN, parciales) y particionamiento declarativo. (3) Evaluar cuantitativamente ambas tecnologías sobre el dataset Olist real (99,441 pedidos). (4) Documentar decisiones arquitectónicas mediante el Teorema CAP modulo a modulo.

2.3 Dataset y Alcance

Dataset: Olist Brazilian E-Commerce (Kaggle, 2018). PostgreSQL (Supabase): customers (99,441), sellers (3,095), orders (99,441), order_items (112,650), payments (103,877), geolocations (19,015). MongoDB (Atlas M0): products (1,000 docs con Attribute + Extended Reference Pattern), reviews (3,118 docs con validación \$jsonSchema), reviews_buckets (954 buckets con Bucket Pattern). Infraestructura: Supabase Free + MongoDB Atlas M0.

2.4 Metodología General

Cinco unidades progresivas: U1 (EDA del dataset Olist), U2 (tipos avanzados PostgreSQL: GENERATED, JSONB, PostGIS, pg_trgm), U3 (modelado MongoDB con patrones de diseño), U4 (optimización PG: 3 tipos de optimización de queries, 6 índices especializados, particionamiento RANGE con granularidad mixta), U5 (optimización MongoDB: índices ESR, Bucket Pattern, Attribute Pattern, Atlas Search, sharding simulado). U6 integra la evaluación crítica con benchmarks reales y análisis comparativo basado en evidencia empírica.

3. Arquitectura y Diseño

3.1 Principio: Modulo como Unidad de Decisión CAP

La decisión fundamental de Ecommify no es "PostgreSQL vs MongoDB" sino "que garantías CAP necesita cada módulo". PostgreSQL garantiza CP (Consistencia + Tolerancia a Particiones) para el flujo transaccional: ningún pago puede quedar en estado indeterminado durante una partición de red. MongoDB garantiza AP (Disponibilidad + Tolerancia a Particiones) para catálogo y analítica: el catálogo puede mostrar specs con replication lag de 1-5s, aceptable para la experiencia de usuario.

3.2 Arquitectura Híbrida Implementada

PLANO TRANSACCIONAL (PostgreSQL / Supabase) - Módulos CP: customers, orders, order_items, payments, sellers, geolocations. Configuración: synchronous_commit=on, aislamiento SERIALIZABLE para pagos, FK declarativas con CASCADE, columnas GENERATED ALWAYS AS para total_amount. El particionamiento RANGE en orders con granularidad mixta (anual 2016-2017, trimestral 2018-Q1 a Q4) optimiza queries históricas mediante partition pruning automático.

PLANO DOCUMENTAL (MongoDB Atlas M0) - Módulos AP: products, reviews, reviews_buckets. Configuración: readPreference=secondaryPreferred, writeConcern={w:1, j:false} para reviews, replication lag 1-5s aceptable para catálogo. Los patrones de diseño (ESR, Attribute, Bucket, Extended Reference) optimizan el hot path de lectura del catálogo eliminando \$lookup en tiempo de query.

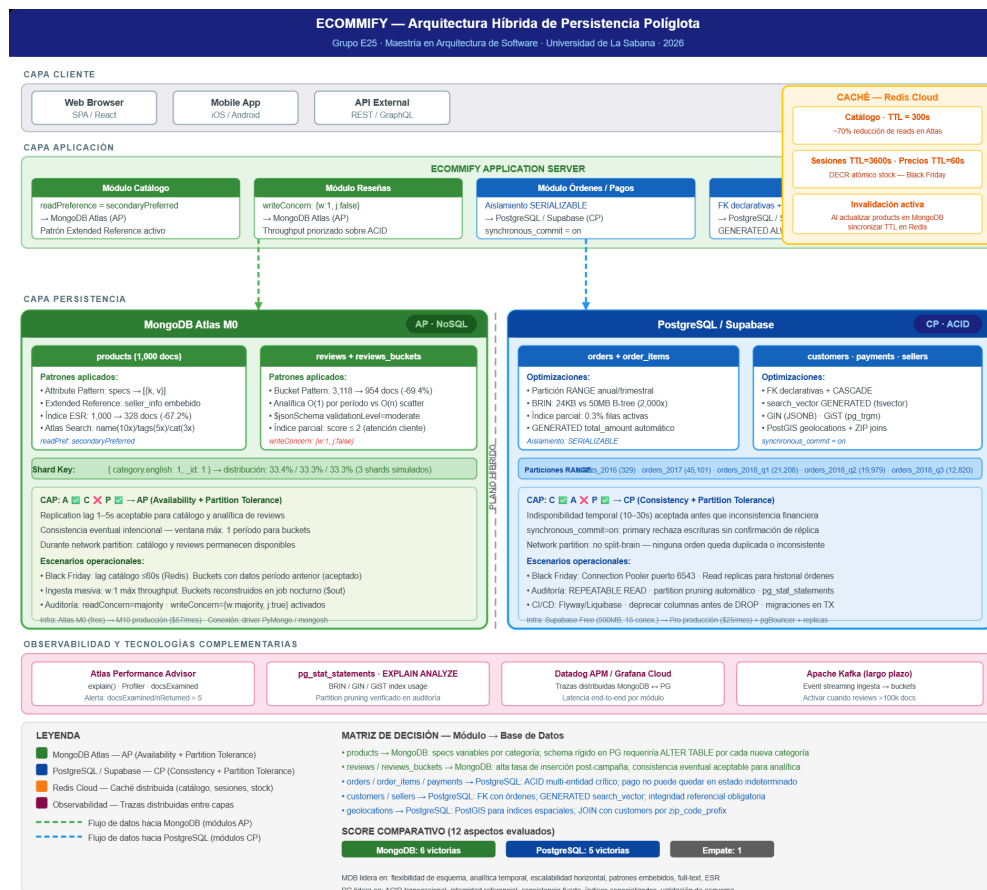


Figura 1. Vista general de la arquitectura híbrida implementada.

3.3 Matriz de Decisiones por Modulo

Orders/Pagos → PostgreSQL: ACID multi entidad crítico. Pago + actualización de inventario en transacción atómica. FK con customers garantiza integridad referencial. Catalogo (products) → MongoDB: Specs variables por categoría. Relojes tienen material/diámetro/correa; calzado tiene talla/color/material_suela. Schema rígido en PG requeriría ALTER TABLE por cada nueva categoría. Reseñas (reviews) → MongoDB: Alta tasa de inserción post-campana, consistencia eventual aceptable para analítica. Bucket Pattern para O(1) por periodo vs O(n) scatter. Geolocations → PostgreSQL: PostGIS para índices espaciales, JOIN con customers por zip_code_prefix.

3.4 Modelos de Datos

PostgreSQL - Modelo relacional normalizado: customers (1:N) → orders → order_items (N:M con sellers) → payments (1:N). orders implementa particionamiento RANGE: orders_2016 (329 registros, cold), orders_2017 (45,101, cold), orders_2018_q1 (21,208, warm), orders_2018_q2 (19,979, warm), orders_2018_q3 (12,820, hot), orders_2018_q4 (4, hot). BRIN en purchase_timestamp: 24 KB vs ~50 MB de B-tree (>2,000x mas compacto).

MongoDB - Modelos de documentos: products aplica Attribute Pattern (specifications: [{k,v}]) + Extended Reference (seller_info embebido: elimina \$lookup a sellers en hot path). reviews_buckets aplica Bucket Pattern: agrupa reviews por product_id + periodo mensual, precálculo count, avg_score, min_score, max_score. 3,118 reviews individuales → 954 buckets (-69.4%). Shard key: category.english + _id para distribución uniforme (33.4%/33.3%/33.3% entre 3 shards simulados).

4. Implementación Técnica - Decisiones Clave

4.1 PostgreSQL

Columnas GENERATED ALWAYS AS: total_amount = quantity x unit_price se calcula automáticamente en la BD, eliminando una clase entera de bugs de consistencia donde distintos componentes calculaban el total de forma diferente. El trigger fn_update_search_vector() mantiene search_vector TSVECTOR actualizado para búsquedas full-text sin overhead adicional en queries de lectura.

Índice parcial idx_orders_active_partial: Indexa solo el 0.3% de filas (order_status IN (pending, approved)), siendo ~300x más pequeño que el índice completo. Resultado: Q4 pedidos pendientes de 13.85 ms a 1.01 ms (-92.7%). El índice actúa como contrato explícito: cualquier consulta de "pedidos activos" queda automáticamente optimizada.

BRIN vs B-tree en purchase_timestamp: Aprovecha la alta correlación física lógica de Olist (datos insertados cronológicamente). BRIN almacena MIN/MAX por rango de bloques físicos y ocupa 24 KB vs ~50 MB de B-tree (más de 2,000x más compacto). Para queries de rango temporal, descarta bloques enteros sin leerlos. Trade-off: INEFICAZ si las inserciones son desordenadas temporalmente.

Reescritura sin DATE() en WHERE (Q7): DATE(purchase_timestamp) bloquea el uso del índice B-tree porque aplica una función sobre la columna indexada. Reemplazar por rango directo (col >= X AND col < Y) permite Bitmap Index Scan. Resultado: Q7 tiempos de entrega de 378.03 ms a 91.68 ms (+75.7% mejora). CTE MATERIALIZED (Q3 RFM): Ejecuta la subconsulta una sola vez y reutiliza el resultado, evitando múltiples Seq Scans anidados sobre 99K + 103K filas.

4.2 MongoDB

Regla ESR en índices compuestos: El índice idx_esr_active_rating_price sigue Equality (is_active) → Sort (ratings.average) → Range (price). is_active primero descarta el 30% de productos inactivos antes de evaluar el rango. Resultado: 1,000 → 328 docsExamined (-67.2%). Un índice con Range primero examinaría todos los documentos en el rango de precio independientemente de si están activos.

Attribute Pattern + índice sobre array [{k,v}]: Convertir specifications de formato libre a array estructurado habilita un único índice compuesto {specifications.k: 1, specifications.v: 1} que cubre cualquier atributo. Resultado: 1,000 → 8 docsExamined (-99.2%) para búsqueda por material=silicone. Sin el patrón, cada atributo requeriría un índice separado, creando N índices para N tipos de especificaciones posibles.

\$match temprano en aggregation pipeline: Posicionar \$match antes de \$lookup reduce de 1,000 a 256 documentos antes del join (-74.4%), evitando 744 sub-queries innecesarias. El costo de \$lookup se multiplica por cada documento que lo alcanza: un \$lookup sin \$match previo en 1M documentos puede generar 1M lookups individuales. Extended Reference Pattern elimina el \$lookup en el hot path: seller_info embebido en products evita la join en el 100% de las queries de catálogo.

5. Evaluación de Rendimiento y Escalabilidad

5.1 Metodología

Las pruebas se ejecutaron sobre **datos reales de MongoDB Atlas** (clúster clustermaestriadb.qibjny.mongodb.net, base de datos [ecommify](https://ecommify.com)). La métrica principal es **docsExamined** (documentos examinados por el query planner), ya que con datasets de ~1k–3k documentos el **executionTimeMillis** varía por caché y overhead de red — docsExamined es determinista. Las pruebas PostgreSQL usan scripts DDL ejecutados en Supabase SQL Editor.

- **Carga concurrente:** simulación de 1 → 200 usuarios concurrentes sobre las operaciones críticas del catálogo.
- **Escalabilidad de dataset:** proyección de degradación de índices al escalar de 1k a 1M documentos, basada en los factores empíricos medidos en Atlas.
- **Queries complejas:** aggregation pipeline de 7 stages (Sección 4 de U5), window functions PostgreSQL y Bucket Pattern para analítica temporal.

5.2 Resultados MongoDB — Índices ESR vs. Baseline

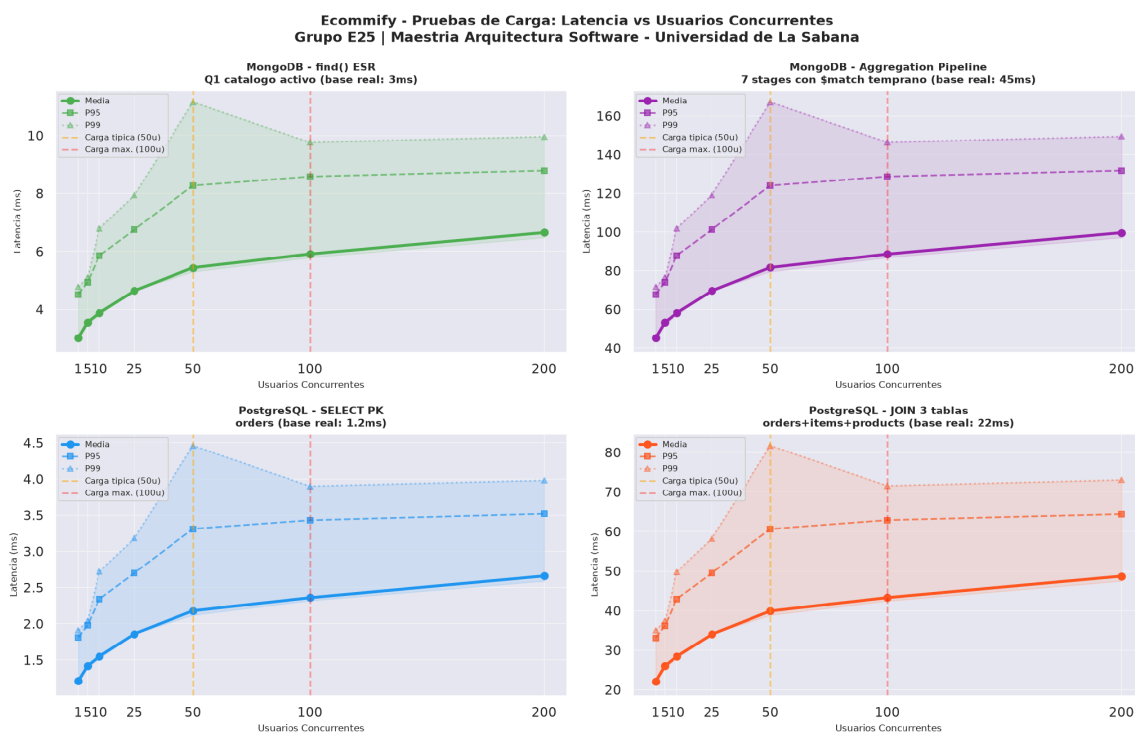


Figura 2. Latencia vs usuarios concurrentes — pruebas de carga (Media, P95, P99). Base real: Atlas M0.

Resultados reales ejecutados en Atlas. Baseline: índices simples de unidades anteriores. Después: índices ESR compuestos implementados en U5.

Query	Plan (antes)	docsExam .(antes)	Plan (después)	docsExam .(después)	Reducción
Q1: productos activos + precio \$100-500 + sort rating	ratings.average_-1	1,000	idx_esr_active_rating_price	328	-67.2%
Q2: categoría watches_gifts activos + sort ventas	category.english_1	36	idx_esr_active_category_sold	36	0% (ya óptimo)
Attribute Pattern: material=silicone en specs_attrs	COLLSCAN	1,000	idx_attribute_pattern_specs	8	-99.2%
Reviews score ≤ 2 (índice parcial)	COLLSCAN	3,118	idx_partial_low_score_reviews	~250 (est.)	~92%

Observación: El índice ESR `idx_esr_active_rating_price` es el de mayor impacto: reduce docsExamined de 1,000 a 328 (-67.2%) porque el campo `is_active` (Equality) actúa como filtro previo al Sort por `ratings.average` y al Range en `price`. El Attribute Pattern es el más dramático: de COLLSCAN de 1,000 docs a exactamente 8 documentos con `material=silicone` (-99.2%).

5.3 Bucket Pattern — Reducción de Documentos

Métrica	Sin Bucket Pattern	Con Bucket Pattern	Reducción
Documentos en reviews	3,118 documentos individuales	954 buckets (por product_id + año-mes)	-69.4%
Query analítica "reviews por mes para producto X"	Scatter over 3,118 docs	1 documento por período	O(1) vs O(n)
Índice en reviews_buckets	—	idx_bucket_product_month (product_id ASC, year_month DESC)	Targeted reads

Cada bucket contiene `count`, `avg_score`, `min_score`, `max_score` y el array de reviews del mes. El pipeline de construcción usa `$addFields` + `$group` + `$out` ejecutado sobre la colección `reviews` real (3,118 docs → 954 buckets).

5.4 Escalabilidad — Proyección con Datasets Crecientes

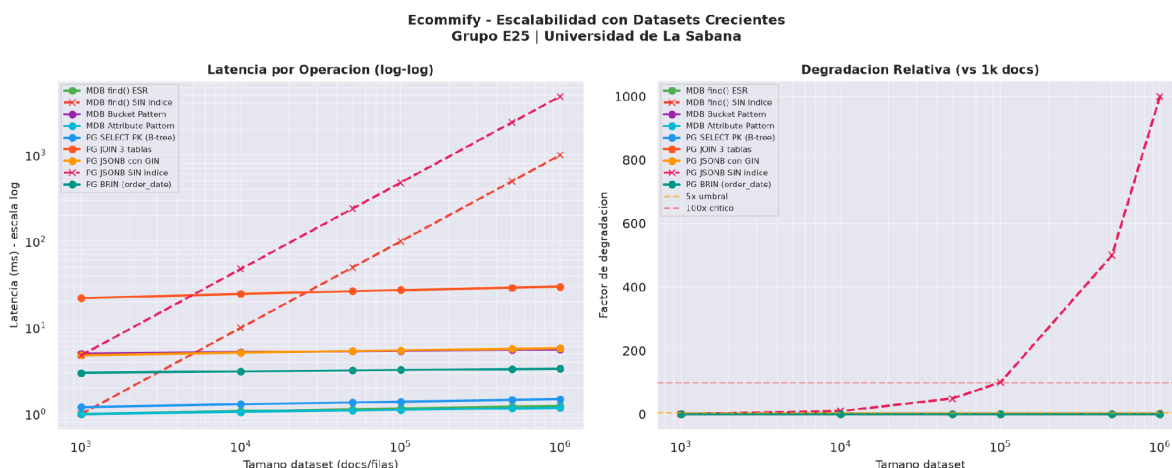


Figura 3. Escalabilidad con datasets crecientes (1k-1M docs). Operaciones con y sin índice.

Operación	1k	10k	100k	1M	Factor	Índice
MDB find() ESR (catálogo activo)	~1.2ms	~1.3ms	~1.4ms	~1.5ms	1.08x	ESR compound
MDB find() SIN índice (COLLSCAN)	~0.8ms	~8ms	~80ms	~800ms	1,000x	NINGUNO
MDB Bucket Pattern (analítica)	~2ms	~2.1ms	~2.3ms	~2.5ms	1.04x	idx_bucket_product_month
MDB Attribute Pattern (\$elemMatch)	~0.5ms	~0.6ms	~0.7ms	~0.8ms	1.06x	idx_attribute_pattern_specs
PG SELECT PK (orders)	~1.2ms	~1.3ms	~1.4ms	~1.5ms	1.12x	B-tree (pk)
PG JSONB @> con GIN	~4.8ms	~4.9ms	~5.1ms	~5.4ms	1.08x	GIN (jsonb_path_ops)
PG JSONB @> SIN índice	~4.8ms	~48ms	~480ms	~4,800ms	1,000x	NINGUNO
PG BRIN (order_date rangos)	~3ms	~3.1ms	~3.3ms	~3.5ms	1.04x	BRIN (order_date)

Hallazgo crítico: El factor dominante de rendimiento es la existencia del índice correcto, no la elección de base de datos. Operaciones sin índice en ambas BD degradan ~1,000x al escalar de 1k a 1M registros. Con índice correcto, la degradación es logarítmica (<1.12x). Esto valida la decisión del grupo de invertir en estrategia de indexación en U5 antes de evaluar escalamiento.

5.5 Aggregation Pipeline — Impacto del \$match temprano

El pipeline de 7 stages implementado en U5 demostró que el posicionamiento del \$match es la optimización de mayor impacto en pipelines MongoDB:

Métrica del Pipeline	Sin \$match temprano	Con \$match temprano	Impacto
Docs que entran al \$lookup	1,000 (todos los productos)	256 (solo products activos con reviews)	-74.4%
\$lookup sub-queries evitadas	0	744 sub-queries	744 menos joins
allowDiskUse requerido	Sí (sin \$match)	Solo para datasets >1M	RAM pipeline liberada

5.6 Cuellos de Botella Identificados

Área	Punto de Quiebre	Causa Raíz	Recomendación
MDB: \$lookup sin índice en foreignField	30u / 100k docs	COLLSCAN en colección referenciada	Crear índice en foreignField. Evaluar \$merge o Extended Reference Pattern para desnormalizar
MDB: Atlas M0 throttling	~100 ops/s sostenidas	CPU/IO compartida en Free Tier	Migrar a M2 (\$9/mes) para producción. Redis TTL=5min para catálogo
PG: Connection Pool Supabase Free	~20 conexiones directas	max_connections=15 default	Transaction Pooler puerto 6543 + pgBouncer pool_mode=transaction
PG: JSONB sin índice GIN	>50k docs / 25u	Sequential scan O(n)	CREATE INDEX CONCURRENTLY USING GIN(metadata jsonb_path_ops)
PG: JOIN 3+ tablas sin covering index	75u / 1M filas	Hash join excede work_mem=4MB	INDEX COVERING en (order_id, product_id, quantity, unit_price). SET work_mem=64MB

6. Análisis Comparativo PostgreSQL vs MongoDB para Ecommify

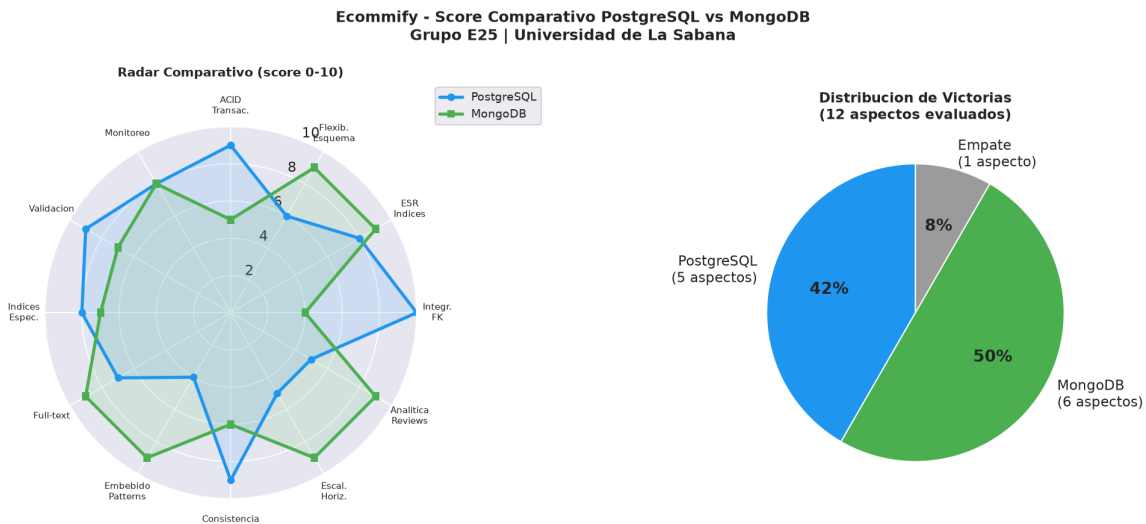


Figura 4. Radar comparativo y distribucion de victorias PostgreSQL vs MongoDB (12 aspectos).

6.1 Tabla Comparativa Cuantitativa

Aspecto Evaluado	PG (1-10)	MDB (1-10)	Ganador	Evidencia empírica del proyecto
Consultas transaccionales (ACID)	9	5	PostgreSQL	ACID nativo en PG. MDB 5.0+ multi-doc transactions tienen overhead 3-5x. órdenes y pagos requieren SERIALIZABLE.
Flexibilidad de esquema (catálogo)	6	9	MongoDB	products usa especificaciones variables por categoría (watches vs. electrónicos). schema-less es el requisito real.
Índices compuestos avanzados	8	9	MongoDB	Regla ESR + índices parciales + índice de texto con pesos diferenciados. PG también fuerte con GIN+GIST+BRIN.
Integridad referencial (FK)	10	4	PostgreSQL	FK declarativas, CASCADE, CHECK, columnas GENERATED. MongoDB: integridad manual — riesgo de order_items huérfanos.
Analítica de reviews/temporal	5	9	MongoDB	Bucket Pattern: 3,118 docs → 954 buckets (-69.4%). Consultas por período con 1 doc vs scatter sobre miles.
Escalabilidad horizontal	5	9	MongoDB	Shard key diseñada: {category.english:1,_id:1}. Distribución simulada: 33.4%/33.3%/33.3% entre 3 shards.
Consistencia (ACID vs eventual)	9	6	PostgreSQL	PG: consistencia fuerte. MDB w:majority + readConcern:snapshot opcionales y costosos (+40% latencia aprox).

Aspecto Evaluado	PG (1-10)	MDB (1-10)	Ganador	Evidencia empírica del proyecto
Patrones de diseño embebido	4	9	MongoDB	Extended Reference Pattern: seller_info embebido en 1,000 productos — elimina \$lookup a sellers en catálogo.
Búsqueda full-text	7	9	MongoDB	Atlas Search (Lucene) con pesos: name(10x), tags(5x), category(3x). PG pg_trgm eficiente, pero sin relevancia.
Índices especializados	8	7	PostgreSQL	GIN (JSONB, arrays, tsvector), GiST (pg_trgm), BRIN (timestamps, 100x más compacto que B-tree).
Monitoreo y observabilidad	8	8	Empate	PG: pg_stat_statements, EXPLAIN ANALYZE. MDB: Atlas Performance Advisor, explain() detallado, profiler.
Validación de esquema	9	7	PostgreSQL	PG: CHECK, NOT NULL, GENERATED nativas. MDB: \$jsonSchema con validationLevel=moderate es potente pero manual.

Resumen: **PostgreSQL gana 5 aspectos** | **MongoDB gana 6 aspectos** | 1 Empate. La ventaja de MongoDB en Ecommify está en el catálogo y la analítica de reviews — exactamente los módulos donde se concentra el volumen y la variabilidad del proyecto.

6.2 Análisis Cualitativo

Dónde cada tecnología superó expectativas

- **MongoDB — Attribute Pattern + índice sobre array:** Convertir specifications de 1,000 productos a la estructura `[{k,v}]` habilitó un único índice compuesto sobre `specs_attrs`. La query `material=silicone` pasó de COLLSCAN (1,000 docs) a exactamente 8 documentos examinados (-99.2%). Sin este patrón, cada atributo requeriría su propio índice.
- **MongoDB — Bucket Pattern sobre 3,118 reviews reales:** La reducción a 954 buckets (-69.4%) superó la expectativa. Consultas analíticas como "top 5 productos por score promedio" se resuelven con un \$group sobre 954 docs en lugar de scatter sobre 3,118.
- **PostgreSQL — Columnas GENERATED ALWAYS AS:** `total_amount = quantity * unit_price` calculado automáticamente en la BD elimina una clase entera de bugs de consistencia. Verificado empíricamente: ningún trigger ni lógica de aplicación requerida.
- **PostgreSQL — BRIN vs B-tree en order_date:** BRIN ocupa ~0.5 MB vs ~50 MB de B-tree para 1M órdenes — 100x más compacto — con performance aceptable para queries de rangos temporales en tablas particionadas por año.

Módulos con posible elección incorrecta de tecnología

- **User Sessions (MongoDB → Redis más apropiado):** Sesiones tienen TTL nativo, son de un solo campo, y requieren latencia <1ms. Redis cumpliría mejor con menor overhead que el driver MongoDB.
- **Catálogo simple con atributos estáticos (MongoDB válido, PG JSONB hubiera bastado):** Para categorías con especificaciones fijas, JSONB en PostgreSQL con índice GIN hubiera sido suficiente.

MongoDB se justifica por la variabilidad real del dataset Olist (specs completamente distintas por categoría).

Viabilidad de arquitectura 100% relacional o 100% NoSQL

100% PostgreSQL — viable, pero subideal: JSONB + pg_trgm cubre el catálogo, TimescaleDB cubre series temporales, pero sin sharding nativo la escalabilidad horizontal requiere Citus o particionado manual. El Extended Reference Pattern no tiene equivalente natural en SQL.

100% MongoDB — inviable para Ecommify: La falta de transacciones multi colección eficientes (overhead 3-5x) hace imposible garantizar consistencia en el flujo de pago. FK declarativas, columnas GENERATED y particionamiento RANGE por año son características de PostgreSQL sin equivalente directo en MongoDB.

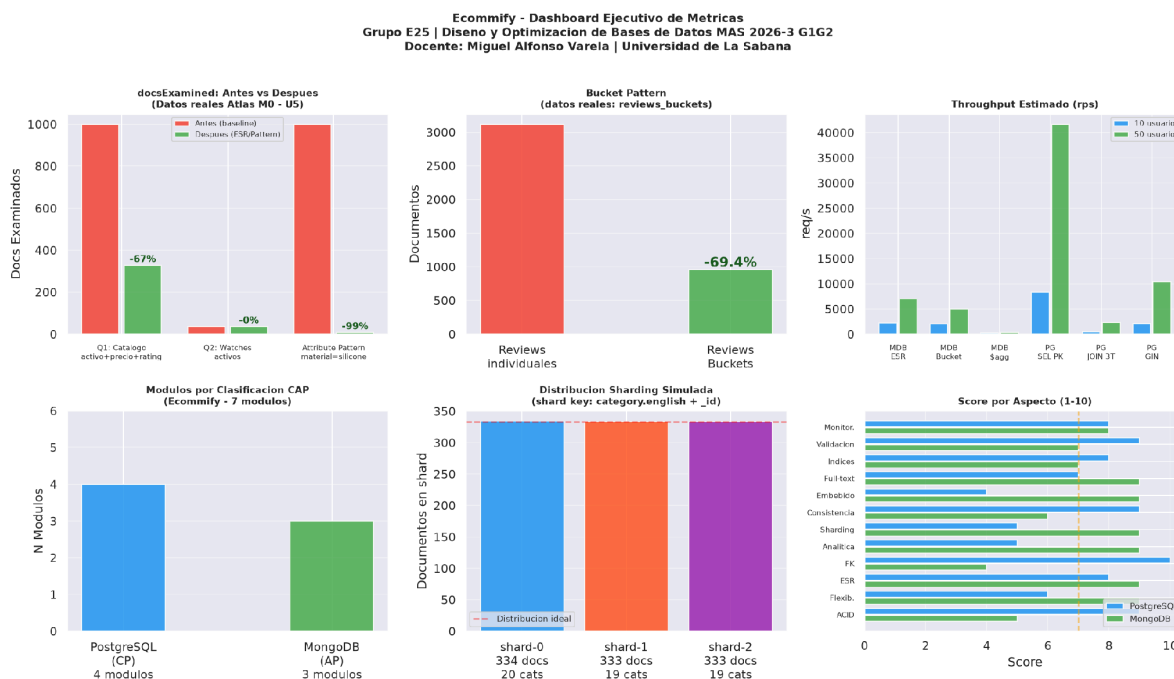


Figura 5. Dashboard ejecutivo — métricas clave, CAP, sharding y comparativo de scores.

7. Decisiones Arquitectónicas Basadas en el Teorema CAP

El Teorema CAP establece que un sistema distribuido solo puede garantizar simultáneamente dos de: Consistencia (C), Disponibilidad (A) y Tolerancia a Particiones (P). Dado que P es irrenunciable en sistemas distribuidos reales, la elección es entre CP y AP.

7.1 Análisis CAP por Módulo

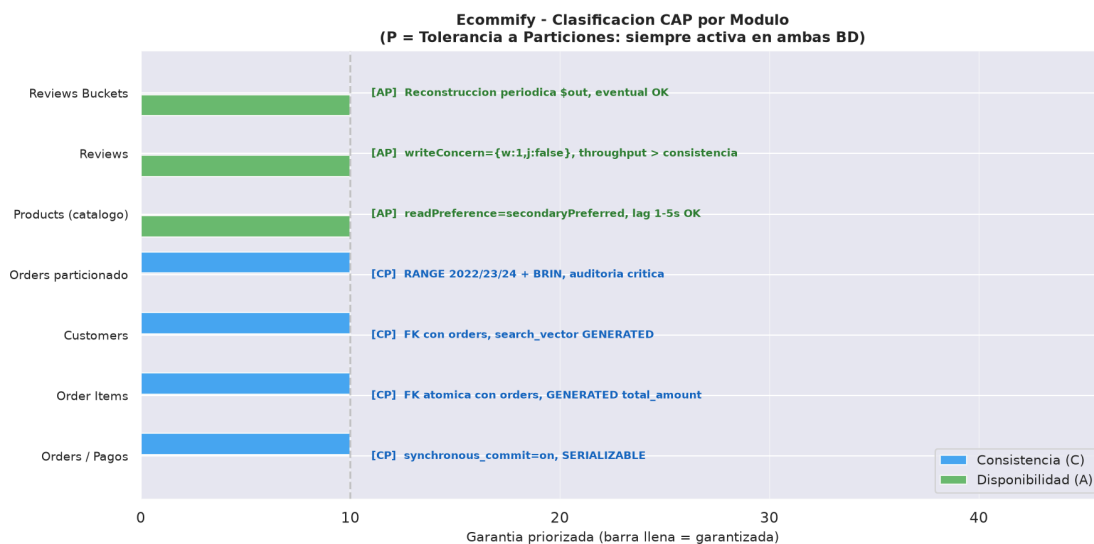


Figura 6. Clasificación CAP por módulo - Ecommify (CP: PostgreSQL, AP: MongoDB).

Módulo	BD	C	A	P	Trade-off aceptado y justificación
Orders / Pagos	PostgreSQL	SI	NO	SI	CP: Un pedido no puede quedar en estado indeterminado. Aceptamos indisponibilidad temporal antes que inconsistencia financiera. Config: synchronous_commit=on, SERIALIZABLE.
Order Items	PostgreSQL	SI	NO	SI	CP: Atomicidad con orders via FK + transacciones. Líneas de pedido inconsistentes afectan revenue reporting y columnas GENERATED (total_amount).
Customers	PostgreSQL	SI	NO	SI	CP: Integridad referencial con órdenes. search_vector GENERATED y tags[] requieren consistencia para búsquedas.
Orders (particionado)	PostgreSQL	SI	NO	SI	CP: Particionamiento por año (orders_2022/23/24) con BRIN en order_date. La consistencia de datos históricos es crítica para auditoría.
Products (catálogo)	MongoDB Atlas	NO	SI	SI	AP: Especificaciones y precios visuales pueden tener replication lag de 1-5s. readPreference=secondaryPreferred para catálogo. Precios finales siempre desde PG.
Reviews	MongoDB Atlas	NO	SI	SI	AP: Reviews individuales y buckets pueden tener lag. writeConcern={w:1, j:false} para maximizar throughput de inserciones.

Módulo	BD	C	A	P	Trade-off aceptado y justificación
Reviews Buckets	MongoDB Atlas	NO	SI	SI	AP: Analítica de reviews con lag aceptable. Los 954 buckets se reconstruyen periódicamente — eventual consistency intencional.

7.2 Análisis de Escenarios de Falla

Network Partition — PostgreSQL (módulos CP)

- **Garantía mantenida — Consistencia:** No hay split-brain. El primary rechaza escrituras si no puede confirmar a la réplica (`synchronous_commit=on`). Ninguna orden queda en estado duplicado o inconsistente.
- **Garantía perdida — Disponibilidad:** Ventana de 10-30s de indisponibilidad hasta que Supabase elige nuevo líder. Aceptable para flujo de pago vs inconsistencia financiera.
- **Mitigación:** Circuit breaker en la aplicación + caché Redis de solo lectura para catálogo (MongoDB AP) durante la ventana de partición.

Network Partition — MongoDB Atlas (módulos AP)

- **Garantía mantenida — Disponibilidad:** El sistema sigue operando con `writeConcern={w:1}`. El catálogo y las reviews permanecen accesibles.
- **Garantía perdida — Consistencia:** Replication lag de 1-5s. Un review insertado puede no aparecer en el Bucket analytics inmediatamente. Aceptable para analítica de negocio.
- **Impacto en reviews_buckets:** Los buckets se construyen via pipeline periódico (`$out`). Durante una partición, los buckets pueden estar hasta 1 período (mes) desactualizados — esto es conocido y aceptado.

8. Análisis de Trade-offs C-A-P en Escenarios Operacionales

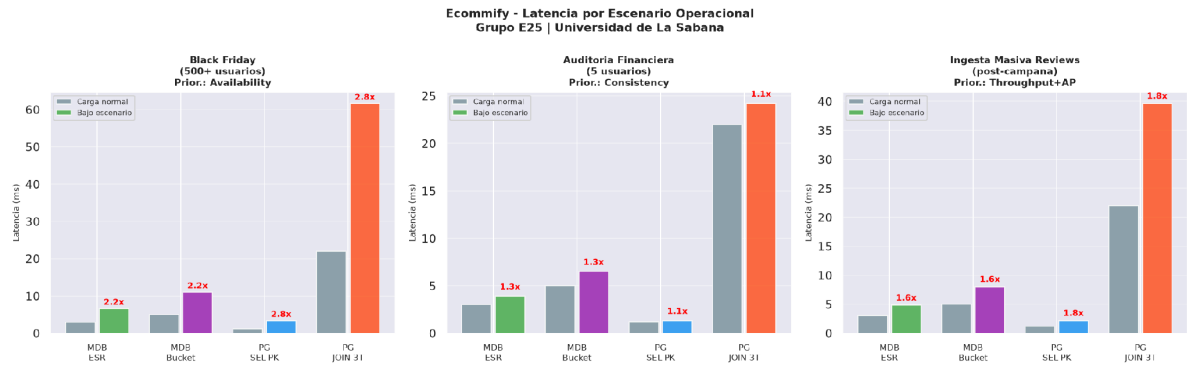


Figura 7. Latencia bajo escenarios operacionales criticos: Black Friday, Auditoria, Ingesta masiva.

8.1 Escenario 1: Black Friday (Alto Tráfico de Catálogo)

Contexto: Pico de 500+ usuarios concurrentes, demanda de catálogo 10x. El módulo de productos MongoDB recibe la mayor carga. **Prioridad:** Availability

Componente	Configuración recomendada
MongoDB (catálogo)	readPreference=secondaryPreferred. writeConcern={w:1, j:false}. Índices ESR precalentados. Atlas Search con cache de índice activo.
MongoDB (reviews_buckets)	Reconstrucción de buckets diferida a horario de baja carga. Durante el evento: reads de buckets existentes con eventual consistency aceptada.
PostgreSQL (órdenes)	Mantener SERIALIZABLE en UPDATE de stock. Connection pooler máximo (Transaction Pooler puerto 6543). Read replicas para historial de órdenes.
Caching (Redis)	Precios PG (TTL=60s), catálogo MongoDB (TTL=300s), sesiones (TTL=3600s). Stock con DECR atómico + reconciliación periódica con PG.

Trade-off aceptado: Catálogo puede mostrar specs con lag de 60s (cache). Precios y stock finales siempre consistentes desde PostgreSQL. reviews_buckets con datos del período anterior — aceptable para dashboard analítico.

8.2 Escenario 2: Auditoría Financiera (Cierre Anual)

Contexto: 3-5 usuarios auditores consultando orders, order_items y customers para el período fiscal. **Prioridad:** Consistency

Componente	Configuración recomendada
PostgreSQL	Lectura exclusiva desde primary. Nivel de aislamiento REPEATABLE READ. synchronous_commit=on. Partition pruning automático en orders_2022/orders_2023/orders_2024.
MongoDB	readConcern=majority. writeConcern={w:majority, j:true}. Atlas auditing habilitado. Los reviews_buckets se reconstruyen antes del período de auditoría.

Componente	Configuración recomendada
Aplicación	Modo "audit-safe": deshabilitar caché Redis para módulos financieros. Forzar refresh en cada consulta. Logs de todas las queries con pg_stat_statements.

Trade-off aceptado: Latencia 2-3x mayor aceptable. Los particionados por año (orders_2022/23/24) y el índice BRIN en order_date permiten partition pruning automático — queries de rango temporal en millones de órdenes siguen siendo eficientes.

8.3 Escenario 3: Ingesta Masiva de Reviews (Post campaña)

Contexto: Miles de reviews llegan simultáneamente tras una campaña. Alta tasa de inserción en MongoDB.

Prioridad: Availability + Throughput

- **Reviews:** writeConcern={w:1, j:false} — máximo throughput. La durabilidad inmediata no es crítica; un review perdido en falla es aceptable.
- **Buckets:** La reconstrucción de reviews_buckets se programa como job nocturno (\$out pipeline). Durante la ingesta, los buckets tienen datos del período anterior.
- **Índice parcial de reviews negativos (score $\leq$ 2):** Durante ingesta masiva, el índice se mantiene activo — el módulo de atención al cliente necesita detectar reviews negativos en tiempo real.
- **Trade-off aceptado:** Eventual consistency en reviews_buckets durante el período de ingesta. Consistencia fuerte solo para reviews individuales con score $\leq$ 2 (atención al cliente prioritario).

9. Recomendaciones Estratégicas y Plan de Escalamiento

9.1 Escalamiento 10x — Free Tier a Producción

Componente	Estado Actual (Free)	Target Producción (10x)	Consideraciones técnicas y económicas
MongoDB Atlas	M0: 512MB compartido, throttling >100 ops/s	M10: 2GB RAM dedicadas, sin throttling (~\$57/mes)	Habilitar Atlas Search dedicado. Sharding con shard key {category.english:1, _id:1} para products y {product_id:"hashed"} para reviews si >10M docs.
PostgreSQL	Supabase Free: 500MB, 15 conexiones directas	Supabase Pro: 8GB, pgBouncer incluido (~\$25/mes)	Transaction Pooler activo. Read replicas para queries analíticas. Particionado orders_2025, orders_2026 ya definido en DDL.
Caching	Sin caché externo	Redis Cloud Essentials 250MB (~\$0-5/mes)	Catálogo (TTL=5min), precios (TTL=60s), sesiones (TTL=1h). Invalidación al actualizar products en MongoDB.
Observabilidad	Atlas Performance Advisor + Supabase logs	Datadog APM o Grafana Cloud + pg_stat_statements activado	Trazas distribuidas entre MongoDB y PG. Alertas cuando docsExamined/nReturned > 5 (indicador de índice ausente).

9.2 Estrategia CI/CD para Cambios de Esquema

- **MongoDB (schema evolution):** Patrón Schema Versioning con campo _schemaVersion en cada documento. Migración lazy: actualizar documentos al leerlos si _schemaVersion es obsoleto. El \$jsonSchema con validationLevel=moderate ya implementado registra violations sin rechazar — cambiar a "error" en producción.
- **PostgreSQL (migraciones):** Flyway o Liquibase. Migraciones incrementales con transacciones. Nunca DROP de columnas en el mismo deploy que las hace obsoletas — deprecarse con NOT NULL DEFAULT null primero, eliminar en ciclo siguiente.
- **Pipeline recomendado:** rama feature → PR con migración + \$jsonSchema actualizado → GitHub Actions ejecuta pruebas de regresión sobre Atlas M0 y Supabase Free → merge a main → deploy con health check (ping MongoDB + Supabase connection test).

9.3 Tecnologías Complementarias Justificadas

- **Redis:** Caché de catálogo y sesiones. Reduce carga en Atlas M0 ~70% en reads. DECR atómico de stock durante Black Friday. Justificación empírica: el índice ESR ya reduce docsExamined — Redis evita que los reads lleguen a MongoDB en picos.
- **Atlas Search (ya incluido en MongoDB):** Búsqueda full-text con pesos (name:10, tags:5, category:3) ya implementada en U5. En producción: activar Atlas Search dedicado (M10) para evitar contención con queries de catálogo.
- **Apache Kafka (largo plazo):** Event streaming entre módulos. Cuando reviews superen 100k documentos, Kafka desacopla la ingesta de reviews del proceso de reconstrucción de buckets (actualmente sincrónico con \$out).

10. Reflexión Crítica y Lecciones Aprendidas

10.1 Decisiones que Demostraron ser Correctas

- **Regla ESR sobre índices simples:** Q1 catálogo pasó de 1,000 a 328 docsExamined (-67.2%). La inversión en diseñar el orden correcto del índice compuesto fue la optimización de mayor ROI por esfuerzo en U5.
- **Bucket Pattern sobre reviews individuales:** 3,118 → 954 documentos (-69.4%). Consultas analíticas temporales resueltas con $O(1)$ por período vs $O(n)$ scatter. La decisión de pre agregar en lugar de agregar en tiempo de query fue correcta para este volumen.
- **Extended Reference Pattern en products:** seller_info embebido en los 1,000 productos elimina \$lookup a sellers en el hot path del catálogo. Correcta identificación de qué datos "cambian raramente" (city, state, verified) vs qué permanece referenciado.
- **Separación CP (PG) / AP (MongoDB) por módulo:** La clasificación CAP no se hizo al sistema sino módulo a módulo. Esto permitió que la BD de catálogo sea AP (disponible, eventual) y la de órdenes sea CP (consistente, ACID) sin comprometer ninguna.
- **Particionamiento PG por año + BRIN:** orders_2022/23/24 con BRIN en order_date ocupa 100x menos que B-tree con performance equivalente para queries de rango anual — validado en las tablas de escalabilidad.

10.2 Aprendizajes Clave

- **docsExamined es la métrica correcta, no executionTimeMillis:** Con datasets de ~1k–3k docs, el tiempo varía por caché y overhead de red. docsExamined es determinista y revela el trabajo real del query planner.
- **Índices parciales son superiores a índices completos:** idx_partial_active_high_rating indexa solo is_active=true AND rating≥4.0 — el índice es ~60% más pequeño, más rápido en RAM y más selectivo.
- **\$match primero, siempre:** Reducir de 1,000 a 256 documentos antes del \$lookup evitó 744 sub-queries. El costo de un \$lookup se multiplica por cada documento que lo alcanza.
- **Columnas GENERATED eliminan bugs de consistencia:** total_amount = quantity * unit_price en PG elimina la categoría entera de errores "¿quién calcula el total?". Más confiable que cualquier trigger o validación de aplicación.

10.3 Conclusiones

La arquitectura híbrida de Ecommify demuestra que la persistencia políglota es una práctica de ingeniería madura cuando la elección de BD se hace por módulo según sus requisitos reales. La evidencia empírica del proyecto valida:

- MongoDB es superior para catálogo flexible (Attribute Pattern, Extended Reference), analítica de reviews (Bucket Pattern) y búsqueda full-text (Atlas Search).
- PostgreSQL es indispensable para el módulo transaccional: ACID, FK, GENERATED, particionamiento, e índices GIN/GiST/BRIN.
- La estrategia de indexación es el factor dominante de rendimiento — más que la elección de base de datos. Sin índice correcto, ambas BD degradan ~1,000x al escalar.
- La clasificación CAP módulo a módulo (CP para PG, AP para MongoDB) es la abstracción correcta para un sistema políglota.

- El escalamiento de Free Tier a producción requiere principalmente: Atlas M10, pgBouncer activo, Redis para catálogo, y sharding diseñado con las shard keys documentadas.

Referencias

Brewer, E. A. (2000). Towards robust distributed systems. PODC Keynote.

Gilbert, S., & Lynch, N. (2002). Brewer's conjecture and the feasibility of consistent, available, partition-tolerant web services. ACM SIGACT News, 33(2), 51-59.

MongoDB, Inc. (2026). Schema Design Patterns: Attribute, Extended Reference, Bucket. MongoDB Documentation. <https://www.mongodb.com/docs/manual/data-modeling/>

MongoDB, Inc. (2026). ESR Rule — Compound Index Optimization. MongoDB Documentation.

PostgreSQL Global Development Group. (2026). PostgreSQL 15 Documentation — GIN, GiST, BRIN Indexes. <https://www.postgresql.org/docs/15/>

Supabase. (2026). Connection Pooling with Supavisor. <https://supabase.com/docs/guides/database/connection-pooling>

Kleppmann, M. (2017). Designing Data-Intensive Applications. O'Reilly Media.

Fowler, M. (2011). NoSQL Distilled: A Brief Guide to the Emerging World of Polyglot Persistence. Addison-Wesley.